MODUL 20 · BAB VI · ALGORITMA GRAF

Algoritma Kruskal untuk Minimum Spanning Tree

Memilih busur dari yang paling ringan, menolak yang membentuk cycle, sampai semua simpul tergabung dalam satu tree.

TUJUAN PEMBELAJARAN

Setelah mempelajari modul ini, kamu diharapkan mampu:

- Menjelaskan langkah-langkah algoritma Kruskal dan peran fungsi MAKE-SET, FIND-SET, dan UNION.
- Menjalankan Kruskal pada graf contoh dan menyusun tabel himpunan busur A serta himpunan simpul setiap langkah.
- Menjelaskan mengapa busur yang diterima Kruskal selalu aman (safe).
- Menganalisis running time $O(E \lg V)$ beserta bagian yang dominan.

PRASYARAT

- Modul 19: konsep MST, cut, light edge, safe edge.
- Modul 08: pengurutan $O(n \lg n)$.

BAB / MATERI ASAL

Materi 12. MINIMUM SPANNING TREE, algoritma Kruskal dan Prim (bagian Algoritma Kruskal)

POSISI DALAM URUTAN BELAJAR

Algoritma MST pertama. Dibandingkan dengan algoritma Prim di Modul 21.

DAFTAR ISI

1. Busur Termurah Lebih Dulu 1
2. Pseudocode Kruskal 2
3. Penelusuran Kruskal pada Graf Contoh 3
4. Kebenaran dan Running Time 5
5. Kesimpulan 6
- Kuis Pemahaman 8
- Kunci Jawaban dan Pembahasan 9
- Referensi 9

PENDAHULUAN

1. Busur Termurah Lebih Dulu

Bayangkan kamu harus menghubungkan sembilan kota dengan kabel dan memiliki daftar semua kemungkinan ruas kabel beserta biayanya. Cara paling sederhana: urutkan ruas-ruas itu dari yang termurah, lalu pasang satu per satu. Setiap kali hendak memasang, periksa dulu: apakah dua kota itu sudah terhubung lewat kabel yang terpasang sebelumnya? Jika sudah, ruas ini tidak diperlukan karena hanya akan membentuk cycle; lewati saja.

Itulah **algoritma Kruskal** (Kruskal, 1956). Algoritma ini Greedy murni: di setiap langkah ia mengambil busur paling ringan yang masih berguna. Selama berjalan, busur-busur terpilih membentuk beberapa tree terpisah (sebuah *forest*) yang lama-kelamaan menyatu menjadi satu MST. Modul ini membahas

pseudocode Kruskal dari bahan kuliah, struktur data himpunan saling lepas yang dibutuhkannya, penelusuran lengkap pada graf contoh, dan analisis running time-nya.

PEMBAHASAN INTI

2. Pseudocode Kruskal

MST-KRUSKAL(G, w)

```
1  A = ∅
2  for each vertex v ∈ G.V
3      MAKE-SET(v)
4  sort the edges of G.E into nondecreasing order by weight w
5  for each edge (u, v) ∈ G.E, taken in nondecreasing order by weight
6      if FIND-SET(u) ≠ FIND-SET(v)
7          A = A ∪ {(u, v)}
8          UNION(u, v)
9  return A
```

Parameter masukan (G, w): G adalah graf tak berarah (himpunan simpul dan himpunan busur), dan w adalah bobot setiap busur.

Langkah	Penjelasan
1	A adalah himpunan busur yang terpilih untuk membangun tree dengan total bobot minimum. Awalnya kosong.
2–3	Untuk setiap simpul, panggil `MAKE-SET(v)`, yaitu membentuk himpunan (tree) yang hanya berisi simpul v. Hasilnya $\|V\|$ tree berukuran satu.
4	Urutkan semua busur berdasarkan bobot dari kecil ke besar.
5	Proses setiap busur sesuai urutan itu. Jika semua busur sudah diproses, lanjut ke langkah 9.
6	Bandingkan `FIND-SET(u)` dan `FIND-SET(v)`: apakah himpunan (tree) yang memuat u berbeda dari yang memuat v? Jika ya, lanjut ke langkah 7; jika tidak, busur ini akan membentuk cycle, sehingga kembali ke langkah 5.
7	Masukkan busur (u, v) ke A.
8	`UNION(u, v)`: gabungkan tree yang memuat u dan tree yang memuat v menjadi satu.
9	Kembalikan A, yaitu himpunan busur yang membentuk MST.

Tabel 1. Penjelasan langkah-langkah algoritma Kruskal.

2.1 Struktur data himpunan saling lepas

Kruskal membutuhkan cara cepat untuk menjawab "apakah u dan v sudah berada di tree yang sama?". Struktur yang dipakai adalah **himpunan saling lepas** (*disjoint-set*), dengan tiga operasi:

- `MAKE-SET(v)`: membuat himpunan baru $\{v\}$.
- `FIND-SET(v)`: mengembalikan **wakil** (representatif) himpunan yang memuat v. Dua simpul berada di himpunan yang sama jika dan hanya jika FIND-SET-nya sama.
- `UNION(u, v)`: menggabungkan himpunan yang memuat u dan himpunan yang memuat v.

Implementasi yang umum menyimpan setiap himpunan sebagai pohon, dengan setiap simpul menunjuk ke induknya dan akar pohon sebagai wakil. Dengan dua teknik sederhana, *union by rank* (pohon yang lebih pendek dijadikan anak) dan *path compression* (simpul pada jalur pencarian langsung diarahkan ke akar), setiap operasi berjalan dalam waktu yang hampir konstan (Cormen dkk., 2009, Bab 21).

CONTOH DAN PENERAPAN

3. Penelusuran Kruskal pada Graf Contoh

Graf contoh sama dengan Modul 19 (9 simpul, 14 busur).

Langkah 1. $A = \emptyset$.

Langkah 2–3. Terbentuk **9 tree** yang masing-masing berisi satu simpul:

$$\{a\},\ \{b\},\ \{c\},\ \{d\},\ \{e\},\ \{f\},\ \{g\},\ \{h\},\ \{i\}.$$

CATATAN KOREKSI TERHADAP SUMBER

Bahan kuliah menulis "hasil langkah 2 dan langkah 3 adalah berupa 8 tree", padahal daftar yang ditulis berisi 9 himpunan. Yang benar adalah **9 tree**, satu untuk setiap simpul.

Langkah 4. Urutan 14 busur berdasarkan bobot (dari bahan kuliah):

$$(h,g),\ (c,i),\ (g,f),\ (a,b),\ (c,f),\ (g,i),\ (c,d),\ (h,i),\ (b,c),\ (a,h),\ (d,e),\ (e,f),\ (b,h),\ (d,f).$$

Busur-busur yang bobotnya sama, seperti (b,c) dan (a,h) yang sama-sama 8, boleh diurutkan bebas di antara sesamanya.

Langkah 5–8. Hasil pemrosesan setiap busur:

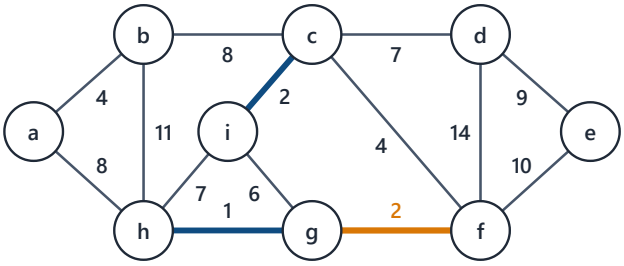
Busur	FIND-SET(u) ≠ FIND-SET(v)?	Himpunan simpul setelah UNION
(h,g) = 1	True	{a}, {b}, {c}, {d}, {e}, {f}, {g,h}, {i}
(c,i) = 2	True	{a}, {b}, {c,i}, {d}, {e}, {f}, {g,h}
(g,f) = 2	True	{a}, {b}, {c,i}, {d}, {e}, {f,g,h}
(a,b) = 4	True	{a,b}, {c,i}, {d}, {e}, {f,g,h}
(c,f) = 4	True	{a,b}, {c,f,g,h,i}, {d}, {e}
(g,i) = 6	False	tidak berubah
(c,d) = 7	True	{a,b}, {c,d,f,g,h,i}, {e}
(h,i) = 7	False	tidak berubah
(b,c) = 8	True	{a,b,c,d,f,g,h,i}, {e}
(a,h) = 8	False	tidak berubah
(d,e) = 9	True	{a,b,c,d,e,f,g,h,i}
(e,f) = 10	False	tidak berubah
(b,h) = 11	False	tidak berubah

(d,f) = 14	False	tidak berubah

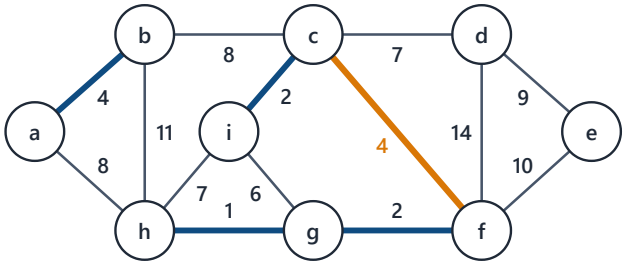
Tabel 2. Hasil algoritma Kruskal langkah 5–8 (sesuai tabel pada bahan kuliah).

Busur yang ditolak semuanya menghubungkan dua simpul yang sudah berada dalam tree yang sama. Misalnya (g, i): setelah (c, f) diterima, g dan i sudah terhubung lewat $g - f - c - i$, sehingga menambahkan (g, i) akan membentuk cycle $g \to f \to c \to i \to g$.

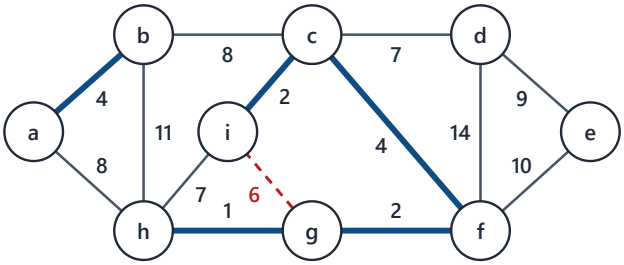
Gambar berikut memperlihatkan perkembangan himpunan A. Busur oranye adalah busur yang baru saja diterima, busur biru tebal adalah busur yang sudah ada di A, dan busur merah putus-putus adalah busur yang baru saja ditolak.



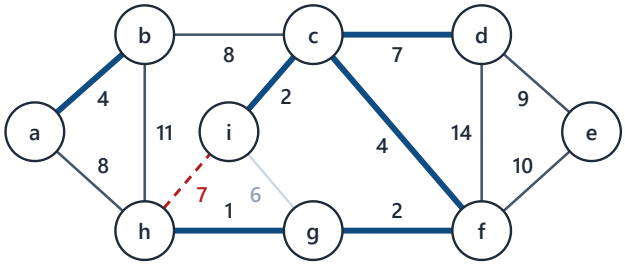
Gambar 1. (a) Setelah memproses (h,g), (c,i), (g,f): ketiganya diterima.



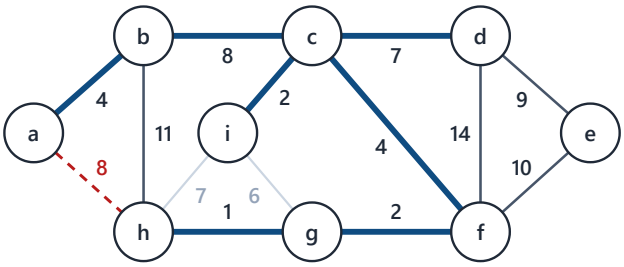
Gambar 2. (b) (a,b) dan (c,f) diterima. (c,f) menyatukan {c,i} dengan {f,g,h}.



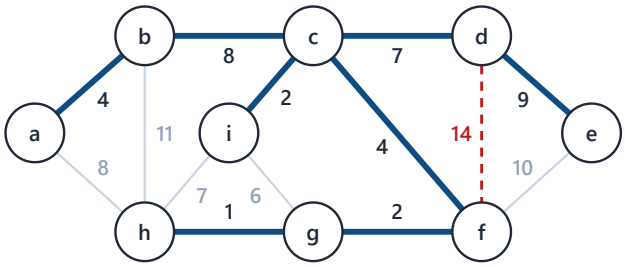
Gambar 3. (c) (g,i) ditolak: g dan i sudah satu tree, akan membentuk cycle.



Gambar 4. (d) (c,d) diterima; (h,i) ditolak.



Gambar 5. (e) (b,c) diterima; (a,h) ditolak karena a dan h sudah terhubung.



Gambar 6. (f) (d,e) diterima; (e,f), (b,h), (d,f) ditolak. MST selesai, bobot 37.

Langkah 9. $A = \{(h, g), (c, i), (g, f), (a, b), (c, f), (c, d), (b, c), (d, e)\}$, dengan total bobot $1 + 2 + 2 + 4 + 4 + 7 + 8 + 9 = 37$. Busur-busur ini sama dengan MST T_1 pada Modul 19, hanya ditulis dalam urutan

berbeda.

PENGARUH URUTAN BUSUR BERBOBOT SAMA

Jika (a, h) diurutkan sebelum (b, c), Kruskal akan menerima (a, h) dan menolak (b, c), sehingga menghasilkan MST T_2 dari Modul 19. Bobotnya tetap 37. Kedua hasil sama benarnya.

3.1 Implementasi Python

Python: Kruskal dengan union-find (union by rank + path compression)

```python
def kruskal(V, busur):
    induk = {v: v for v in V}                  # MAKE-SET untuk setiap simpul
    rank = {v: 0 for v in V}
    def find_set(x):
        if induk[x] != x:
            induk[x] = find_set(induk[x])      # path compression
        return induk[x]
    def union(x, y):
        rx, ry = find_set(x), find_set(y)
        if rank[rx] < rank[ry]:
            rx, ry = ry, rx
        induk[ry] = rx                         # union by rank
        if rank[rx] == rank[ry]:
            rank[rx] += 1
    A = []
    for u, v, w in sorted(busur, key=lambda e: e[2]):   # sorted() stabil
        if find_set(u) != find_set(v):
            A.append((u, v, w))
            union(u, v)
    return A

urutan_bahan = [("h","g",1), ("c","i",2), ("g","f",2), ("a","b",4), ("c","f",4),
                ("g","i",6), ("c","d",7), ("h","i",7), ("b","c",8), ("a","h",8),
                ("d","e",9), ("e","f",10), ("b","h",11), ("d","f",14)]
A = kruskal("abcdefghi", urutan_bahan)
print([u + v for u, v, _ in A], sum(w for *_, w in A))

ah_dulu = [e for e in urutan_bahan if e[:2] != ("b","c")]
ah_dulu.insert(ah_dulu.index(("a","h",8)) + 1, ("b","c",8))   # (a,h) sebelum (b,c)
A = kruskal("abcdefghi", ah_dulu)
print([u + v for u, v, _ in A], sum(w for *_, w in A))
```

Keluaran program

```
['hg', 'ci', 'gf', 'ab', 'cf', 'cd', 'bc', 'de'] 37
['hg', 'ci', 'gf', 'ab', 'cf', 'cd', 'ah', 'de'] 37
```

ANALISIS DAN HUBUNGAN KONSEP

4. Kebenaran dan Running Time

4.1 Mengapa Kruskal benar?

Misalkan Kruskal hendak menerima busur (u, v), dan C adalah tree (di forest A) yang memuat u. Perhatikan cut $(C, V - C)$. Cut ini **respects** A, karena tidak ada busur A yang keluar dari C. Busur (u, v) menyeberangi cut ini (karena v berada di tree lain). Semua crossing edge yang lebih ringan sudah diproses lebih dulu dan pasti ditolak, sebab jika diterima, ujungnya tidak lagi berada di luar C. Jadi (u, v) adalah **light edge** untuk cut itu, dan menurut Teorema 23.1 (Modul 19), (u, v) adalah **safe edge**. Setiap busur yang diterima aman, sehingga hasil akhirnya MST.

4.2 Running time

Bagian	Biaya
Langkah 2–3: $\|V\|$ kali MAKE-SET	$O(V)$
Langkah 4: mengurutkan $\|E\|$ busur	$O(E \lg E)$
Langkah 5–8: $O(E)$ operasi FIND-SET dan UNION	$O(E\,\alpha(V))$, dengan α fungsi yang tumbuh sangat lambat (praktis ≤ 4)

Tabel 3. Biaya setiap bagian algoritma Kruskal (dengan union-find yang efisien).

Bagian yang dominan adalah **pengurutan**. Karena graf terhubung memiliki $|E| \geq |V| - 1$ dan $|E| < |V|^2$, berlaku $\lg E = O(\lg V)$. Jadi running time Kruskal adalah $O(E \lg V)$ (Cormen dkk., 2009, Bab 23.2).

4.3 Hubungan dengan modul lain

- Kruskal adalah algoritma Greedy (Modul 13) yang pilihan greedy-nya adalah busur teringan yang tidak membentuk cycle.
- Pola "gabungkan dua bagian terkecil" mirip Huffman (Modul 15), yang menggabungkan dua simpul berfrekuensi terkecil.
- Prim (Modul 21) menghasilkan MST dengan bobot yang sama, tetapi menumbuhkan satu tree dari sebuah akar dan memakai antrean prioritas.
- Jika graf tidak terhubung, Kruskal tetap berjalan dan menghasilkan *minimum spanning forest*.

KESIMPULAN

5. Kesimpulan

- Kruskal mengurutkan semua busur dari bobot terkecil, lalu menerima setiap busur yang menghubungkan dua tree berbeda dan menolak busur yang akan membentuk cycle.
- MAKE-SET, FIND-SET, dan UNION (himpunan saling lepas) dipakai untuk mendeteksi apakah dua simpul sudah satu tree.
- Pada graf contoh, Kruskal menerima (h,g), (c,i), (g,f), (a,b), (c,f), (c,d), (b,c), (d,e) dengan total bobot 37.
- Setiap busur yang diterima adalah light edge untuk cut antara tree-nya dan sisa graf, sehingga safe.
- Running time $O(E \lg V)$, didominasi oleh pengurutan busur.

Kuis Pemahaman

Kerjakan semua soal terlebih dahulu tanpa melihat kunci jawaban. Kunci jawaban beserta pembahasannya ada di halaman berikutnya.

1 PILIHAN GANDA

Pada Kruskal, busur (u, v) ditolak jika ...

A. Bobotnya lebih besar dari busur sebelumnya.
B. FIND-SET(u) = FIND-SET(v), yaitu u dan v sudah berada di tree yang sama.
C. Salah satu simpulnya sudah pernah dipakai.
D. A sudah berisi lebih dari $|V|/2$ busur.

2 PENELUSURAN

Jalankan Kruskal pada graf dengan busur $(p, q) = 3$, $(p, r) = 1$, $(q, r) = 2$, $(q, s) = 5$, $(r, s) = 4$, $(s, t) = 6$, $(r, t) = 7$. Tuliskan busur yang diterima/ditolak dan total bobot MST.

3 ANALISIS

Pada penelusuran di modul ini, jelaskan cycle apa yang akan terbentuk jika busur (h, i) diterima.

4 BENAR/SALAH DENGAN ALASAN

"Kruskal dapat berhenti lebih awal begitu A berisi $|V| - 1$ busur." Benar atau salah?

5 PERHITUNGAN

Sebuah graf memiliki $|V| = 1.000$ dan $|E| = 50.000$. Perkirakan banyaknya operasi untuk tahap pengurutan dan untuk tahap pemrosesan busur. Bagian mana yang dominan?

6 ANALISIS KASUS

Setelah Kruskal selesai pada graf contoh, bobot busur (e, f) diturunkan dari 10 menjadi 5. Busur mana yang akan berubah status? Berapa bobot MST yang baru?

7 PILIHAN GANDA

Selama Kruskal berjalan, himpunan A selalu berbentuk ...

A. Satu tree yang terhubung.
B. Forest (kumpulan tree) yang tidak memiliki cycle.
C. Graf yang mungkin memiliki cycle.
D. Lintasan dari a ke e.

Kunci Jawaban dan Pembahasan

Bandingkan jawabanmu, lalu baca pembahasannya untuk memahami alasan di balik setiap jawaban.

Soal 1

Jawaban: B. Busur ditolak hanya jika kedua ujungnya sudah satu tree, karena menambahkannya membentuk cycle. Bobot selalu tidak menurun karena busur diproses terurut (A bukan alasan), dan sebuah simpul boleh dipakai berkali-kali (C salah).

Soal 2

Urutan: $(p, r) = 1$ diterima; $(q, r) = 2$ diterima; $(p, q) = 3$ **ditolak** (p, q sudah terhubung lewat r); $(r, s) = 4$ diterima; $(q, s) = 5$ **ditolak**; $(s, t) = 6$ diterima; $(r, t) = 7$ **ditolak**. MST: $\{(p, r), (q, r), (r, s), (s, t)\}$, total bobot $1 + 2 + 4 + 6 = \mathbf{13}$.

Soal 3

Saat (h, i) diproses, h dan i sudah terhubung lewat $h - g - f - c - i$. Menambahkan (h, i) membentuk cycle $h \to g \to f \to c \to i \to h$.

Soal 4

Benar. Tree dengan $|V|$ simpul memiliki tepat $|V| - 1$ busur, jadi setelah busur ke-$(|V| - 1)$ diterima, semua busur berikutnya pasti ditolak. Pada contoh, setelah (d, e) diterima, sisa busur (e, f), (b, h), (d, f) tidak perlu diperiksa. Penghentian dini ini tidak mengubah orde running time karena pengurutan tetap dominan.

Soal 5

Pengurutan: sekitar $E \lg E = 50.000 \times \lg 50.000 \approx 50.000 \times 15{,}6 \approx 780.000$ langkah. Pemrosesan busur: sekitar $E \cdot \alpha(V) \leq 50.000 \times 4 = 200.000$ langkah. Bagian yang **dominan adalah pengurutan**.

Soal 6

Dengan bobot 5, (e, f) diproses setelah $(c, f) = 4$ dan sebelum $(g, i) = 6$. Saat itu e masih sendirian, sehingga (e, f) **diterima**. Selanjutnya $(d, e) = 9$ akan **ditolak**, karena d sudah terhubung dengan e lewat $d - c - f - e$. MST baru berbobot $37 - 9 + 5 = \mathbf{33}$.

Soal 7

Jawaban: B. A dimulai sebagai $|V|$ tree berukuran satu dan tidak pernah memuat cycle; tree-tree itu menyatu sedikit demi sedikit hingga menjadi satu tree di akhir.

Referensi

Sumber utama: Materi 12. MINIMUM SPANNING TREE, algoritma Kruskal dan Prim (bagian Algoritma Kruskal) (bahan kuliah Perancangan dan Analisis Algoritma). Penjelasan tambahan, koreksi, dan pengayaan mengacu pada pustaka berikut.

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). Cambridge, MA: MIT Press. ISBN 978-0-262-03384-8.
2. Kruskal, J. B. (1956). On the Shortest Spanning Subtree of a Graph and the Traveling Salesman Problem. *Proceedings of the American Mathematical Society*, 7(1), 48–50. https://doi.org/10.1090/S0002-9939-1956-0078686-7

3. Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Upper Saddle River, NJ: Addison-Wesley.